

Under the Hood: Building Multiple Linear Regression from Scratch

Translating Mathematical Theory into a Custom Python Class

THEORY

THEORY

$$\beta = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}$$

PRACTICE

PRACTICE

```
class MeraLR:
    def fit(self, X, y):
        # ... implementation
```


The Raw Material: The Diabetes Dataset

Understanding the Matrix Dimensions

442 Patients (Samples)	10 Features									
	AGE	SEX	BMI	BP	S1	S2	S3	S4	S5	S6
	0.038076	-0.044642	0.050680	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019908	-0.017646
	0.038076	-0.044642	0.050680	0.021872	-0.044223	-0.034821	-0.043401	0.002592	0.019908	-0.017646
	0.038076	-0.044642	0.050680	0.021872	-0.044223	-0.034821	-0.043401	0.002592	0.019908	-0.017646
	...	...	...	...	...	...	...	...	...	...
	0.038076	-0.044642	0.050680	0.021872	-0.044223	-0.034821	-0.043401	0.002592	0.019908	-0.017646
	0.038076	-0.044642	0.050680	0.021872	0.021823	-0.034821	0.043401	0.002592	0.019908	-0.017646
	0.038076	-0.044642	0.050680	0.021872	-0.044223	-0.034821	-0.043401	0.002592	0.019908	-0.017646
	...	...	...	...	...	...	...	...	...	...

Matrix Shape: (442, 10)

```
import numpy as np
from sklearn.datasets import load_diabetes

X, y = load_diabetes(return_X_y=True)
print(X.shape) # Output: (442, 10)
print(y.shape) # Output: (442,)
```

Target (y) = Disease progression score one year after baseline.

The Industry Standard

Current State: Using Scikit-Learn

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
```

1. Split Data

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=2)
```

2. Initialize and Train

```
lr = LinearRegression()
lr.fit(X_train, y_train)
```

The Black Box: All math happens here.

3. Predict and Score

```
y_pred = lr.predict(X_test)
print(r2_score(y_test, y_pred))
```

R2 Score
0.4399

Extracting the Ground Truth

The internal parameters we must replicate exactly.

INTERCEPT (β_0)

151.8833

The bias term.

COEFFICIENTS ($\beta_1 \dots \beta_{10}$)

[-9.158...
-240.17...
520.36...
324.39...
-792.18...
476.74...
101.04...
177.06...
751.27...
67.62...]



**TARGET
ACQUIRED**

The Blueprint: The Normal Equation

Vector of Coefficients (11 values)

Inverse of the Matrix

$$\beta = (X^T X)^{-1} \cdot X^T Y$$

Input Data Matrix

Transpose of Matrix

Target Vector (Output)

Mathematical Logic: This closed-form solution calculates the optimal weights for every input feature simultaneously by minimizing the sum of squared errors.

The Intercept Trick

Why we need to modify the input matrix.

feat_1	feat_2	...	feat_10
		...	
		...	
		...	
		...	
		...	
		...	
		...	

Shape: (442, 10)

Before

`np.insert(X, 0, 1, axis=1)`

$x_0 = 1$
(Bias)

	feat_1	feat_2	...	feat_10
1			...	
1			...	
1			...	
1			...	
1			...	
1			...	
1			...	

Shape: (442, 11)

After

The linear equation $y = \beta_0 + \beta_1 x_1 \dots$ requires a coefficient β_0 . To solve this using matrices, we artificially create a feature column x_0 that always equals 1.

The Toolset: Mapping Math to NumPy

Translation Guide

Operation	Math Symbol	NumPy Syntax
Matrix Inverse	A^{-1}	<code>np.linalg.inv(A)</code>
Dot Product	$A \cdot B$	<code>np.dot(A, B)</code>
Transpose	X^T	<code>X.T</code>
Add Column	$[1 \mid X]$	<code>np.insert(X, 0, 1, axis=1)</code>

Direct translation

The compuner's gide to termmmersion syntax, conifue a- incvading a NumPy syntax, versue a column apindled n to simplated by one... direct translation..

Step 1: Defining the Class Structure

```
1 class MeraLR:
2
3     def __init__(self):
4         self.coef_ = None
5         self.intercept_ = None
6
7     def fit(self, X_train, y_train):
8         '''Logic to train the model'''
9         pass
10
11     def predict(self, X_test):
12         '''Logic to predict on new data'''
13         pass
```

Storage variables for our results (Beta).

Will solve the Normal Equation.

Will apply $y = mx + c$.

Step 2: The 'fit' Method - Preprocessing

```
1  
2  
3  
4  
5  
6  
7 def fit(self, X_train, y_train):  
8     # Insert column of 1s at index 0  
9     X_train = np.insert(X_train, 0, 1, axis=1)  
10
```



This single line transforms our (353, 10) training data into a (353, 11) matrix, ready for the dot product.

Step 3: Calculating the Coefficients

The One-Liner Solution

$$\beta = (X^T X)^{-1} X^T Y$$

```
1  
2  
3  
4  
5  
6  
7 # Calculate the coefficients  
8  
9
```

```
betas = np.linalg.inv(np.dot(X_train.T, X_train)).dot(X_train.T).dot(y_train)
```

$(X^T X)$

Inverse

X^T

Y

Result: Array of 11 values.

Step 4: Storing the Results

Splitting the Beta Vector



```
# The first value is the intercept  
self.intercept_ = betas[0]
```

```
# The rest are the coefficients  
self.coef_ = betas[1:]
```


Step 5: The 'predict' Method

Applying the weights to new data.

Note: We do NOT insert a column of 1s here. Instead, we use the standard linear formula ($y = mx + c$) by adding the intercept scalar at the end.

```
1 def predict(self, X_test):
2     #  $y = X\_test \cdot coef + intercept$ 
3     y_pred = np.dot(X_test, self.coef_) + self.intercept_
4
5     return y_pred
```

mx
(dot product)

$+ c$
(intercept)

Running the Custom Engine

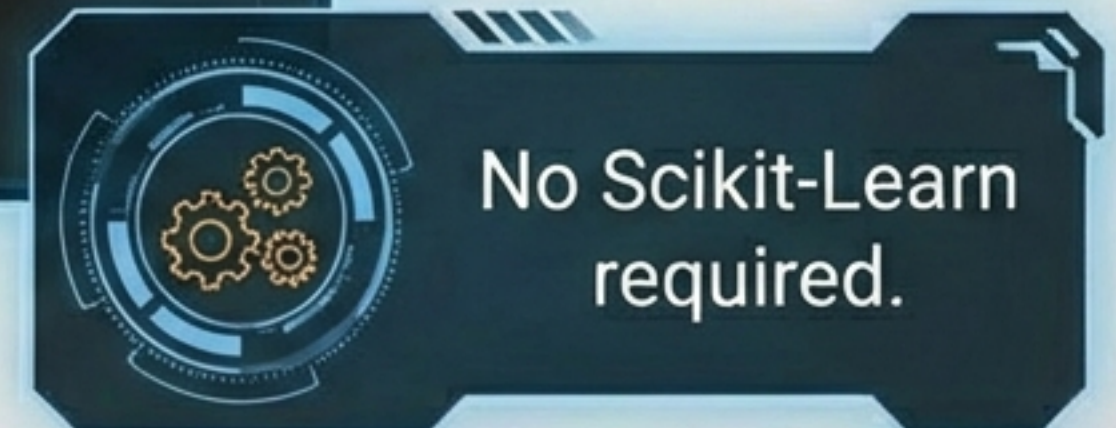
A Drop-in Replacement

```
import MeraLR

# Initialize Custom Class
lr = MeraLR()

# Train
lr.fit(X_train, y_train)

# Predict
y_pred = lr.predict(X_test)
```



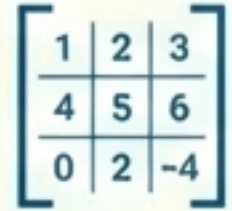
Validation: The Moment of Truth

Metric	Scikit-Learn (Ground Truth)	MeraLR (Our Code)	Status
Intercept	151.8833...	151.8833...	MATCH ✓
Coef [0]	-9.158...	-9.158...	MATCH ✓
Coef [9]	67.62...	67.62...	MATCH ✓
R2 Score	0.4399	0.4399	MATCH ✓

Bit-for-bit identical results using raw NumPy linear algebra.

Mission Accomplished

Math Translated to Code



1	2	3
4	5	6
0	2	-4

Deconstructed the matrix operations.



Built a custom class from scratch.



Validated against industry standard.

Challenge: Try this 'MeraLR' class on a simple dataset with just 1 input feature. The math holds universal.

